Java Core — Lộ trình rút gọn 2 tuần (học kèm AI)

Tổng: 26 bài cũ (đánh số giữ nguyên) + 4 bài mới (34–37) | **Thời gian:** 26/10 – 06/11/2026, 10 ngày học (thứ Hai – thứ Sáu, nghỉ thứ Bảy và Chủ nhật) | **Nhịp:** từ 4 giờ/ngày
Dành cho người đã làm frontend: lướt nhanh phần cú pháp, dồn thời gian vào OOP, Collections, Stream, Concurrency, những phần cần để **đọc hiểu và review code** (kể cả code do AI viết).

📍 Các mốc

T3 27/10	Xong OOP cơ bản: Student, BankAccount, Shape, Comparator, Exception (bài 7–11, 13)
T4 28/10	Xong OOP và Java hiện đại: Enum, Lambda, Record, Sealed (bài 12, 14–16)
T5 29/10	Xong Collections (bài 17–23)
T2 02/11	Xong Stream API (bài 25–30)
T4 04/11	Xong Concurrency, JVM và LRU Cache (bài 32, 34–37)
T6 06/11	Xong dự án cuối (bài 33) và tự kiểm tra theo checklist
T2 09/11	Bắt đầu Spring Boot

Ba cách làm bài

Mỗi bài được gắn một ký hiệu cho biết làm theo cách nào:

Ký hiệu	Cách làm	Dùng AI thế nào
⚡ **Lướt**	Đọc yêu cầu, gõ nhanh, so sánh với JS/TS	Thoải mái hỏi AI "trong Java viết thế nào", miễn là bạn hiểu
🖐 **Tự gõ**	Tự làm từ đầu đến cuối, **không mở AI** cho đến khi xong	Xong rồi mới đưa AI review (prompt bên dưới)
🔍 **Review ngược**	Để AI viết lời giải, bạn làm người review	Tìm lỗi, viết test chứng minh, sửa lại

Prompt review (dùng cho bài 🖐):

```
Review đoạn Java này như một senior backend reviewer. Chỉ ra:
1) bug hoặc edge case chưa xử lý, 2) chỗ chưa đúng idiom Java hiện đại (Java 21),
3) vấn đề hiệu năng/độ phức tạp. Không viết lại toàn bộ, chỉ chỉ ra vấn đề và gợi ý.
```

Quy trình cho bài 🔍:

1. Nhờ AI viết lời giải.
2. Tự đọc và trả lời trước: code xử lý input rỗng hoặc `null` chưa? Xử lý exception đúng chưa? Độ phức tạp bao nhiêu?
3. Viết ít nhất 3 test JUnit, trong đó ít nhất 1 test cho edge case.
4. **Nâng cao:** nhờ AI *cố tình cài 2 bug khó thấy* vào lời giải, rồi bạn tự tìm ra. Đây là cách luyện kỹ năng review nhanh nhất.

> LƯU Ý
> Với bài 🖐: bí quá 30 phút mới hỏi AI, và chỉ hỏi **gợi ý hướng làm**, không xin code. Sau đó tự gõ lại từ đầu.

🟦 PHẦN 1 — Cú pháp Java cho người từ JS (T2 26/10)

Gộp bài 1–6. Mục tiêu không phải luyện logic (bạn đã biết rồi) mà là nắm **Java khác JS ở đâu**.

Bài 1–6 gộp — Bài khởi động ⚡ 📅 T2 26/10

Làm nhanh 3 bài, mỗi bài khoảng 30–40 phút:

- **Bài 2:** in số nguyên tố từ 2 đến N, tính tổng.
- **Bài 5:** xử lý String: đảo chuỗi, kiểm tra palindrome, viết hoa chữ đầu mỗi từ, đếm ký tự.
- **Bài 6:** ma trận NxM ngẫu nhiên: tổng hàng/cột, phần tử lớn nhất, chuyển vị.

Các bài 1, 3, 4 chỉ cần đọc qua. Riêng Bubble Sort ở bài 4 không cần tự viết, chỉ cần biết dùng `Arrays.sort()` và `Arrays.binarySearch()`.

Ghi chú "Java ≠ JS" (tự viết ra một file note, đây là phần quan trọng nhất của ngày 1):

- Kiểu tĩnh; `int` khác `Integer` (primitive và wrapper, autoboxing)
- `==` khác `.equals()` với String và object
- String là immutable; nối chuỗi trong vòng lặp thì dùng `StringBuilder`
- Mảng có kích thước cố định; cần mảng động thì dùng `ArrayList`
- Chia số nguyên: `5 / 2 == 2`
- `switch` expression (`->`) và `var`

Kiến thức: cú pháp cơ bản, `String`, `StringBuilder`, `int[][]`, `Arrays`

🟩 PHẦN 2 — OOP (26/10 – 28/10)

Bài 7 — Class Student 🖐 ⭐⭐ 📅 T2 26/10

Tạo class `Student` (id, name, age, GPA):

- Constructor đầy đủ và constructor mặc định
- Getter/setter có validation (age: 16–30, GPA: 0.0–4.0)
- `toString()`, `equals()` và `hashCode()` so sánh theo id
- Bỏ 2 sinh viên trùng id vào `HashSet`, so sánh kết quả khi có và khi thiếu `hashCode()`

> **Bẫy hay gặp (kể cả AI):** override `equals()` mà quên `hashCode()`; tính `hashCode()` từ field có thể thay đổi giá trị.

Kiến thức: đóng gói, `@Override`, hợp đồng `equals` / `hashCode`

Bài 8, 9, 10 gộp — BankAccount, Shape, Interface ⚡ ⭐⭐ 📅 T3 27/10

Làm nhanh, mỗi phần khoảng 30 phút:

- **Bài 8:** `BankAccount` có `deposit()` / `withdraw()` kèm validation, lịch sử giao dịch, static field `totalAccounts`. *Giữ lại code này để dùng ở bài 34.*
- **Bài 9:** abstract class `Shape` với các lớp con `Circle`, `Rectangle`, `Triangle`; tính tổng diện tích từ một mảng `Shape[]`.
- **Bài 10:** interface `Chargeable` và `Connectable`; method `chargeAll(Chargeable[])`.

Câu cần tự trả lời được: *khi nào dùng abstract class, khi nào dùng interface?*

Kiến thức: `static`, `abstract`, `interface`, polymorphism

Bài 11 — Comparable & Comparator 🖐 ⭐⭐⭐ 📅 T3 27/10

Class `Product implements Comparable<Product>` để so sánh theo giá. Sắp xếp `List<Product>`; thêm `Comparator` để sắp xếp theo tên, theo stock, và theo nhiều tiêu chí: `Comparator.comparing(...).thenComparing(...).reversed()`.

> **Bẫy hay gặp:** viết `compareTo` bằng phép trừ (`a.price - b.price`), dễ bị tràn số hoặc ép kiểu sai. Hãy dùng `Double.compare()` hoặc `Integer.compare()`.

Kiến thức: `Comparable`, `Comparator`, `List.sort()`

Bài 12 — Enum: OrderStatus 🖐 ⭐⭐⭐ 📅 T4 28/10

Enum `OrderStatus` (PENDING, CONFIRMED, SHIPPED, DELIVERED, CANCELLED):

- Có field `description` (mô tả tiếng Việt)
- Có method `canTransitionTo(OrderStatus next)` để kiểm tra chuyển trạng thái hợp lệ
- Class `Order` có `updateStatus()`, ném exception khi chuyển trạng thái sai

> **Bẫy hay gặp:** dựa vào `ordinal()` để so sánh trạng thái; chỉ cần đổi thứ tự khai báo enum là hỏng.

Kiến thức: enum có field/method, state machine (rất hay gặp trong backend thực tế)

Bài 13 — Exception: Máy tính 🖐 ⭐⭐⭐ 📅 T3 27/10

Class `Calculator` với các custom exception: `DivisionByZeroException`, `NegativeSqrtException`, `InvalidOperationException`. Nhập biểu thức như `"10 / 0"`, xử lý và in kết quả hoặc thông báo lỗi cụ thể.

Câu cần tự trả lời được: *checked và unchecked exception khác nhau thế nào? Custom exception của bạn nên extends class nào, vì sao?*

> **Bẫy hay gặp:** `catch (Exception e)` chung chung rồi nuốt lỗi; `finally` có `return`; mất stack trace khi wrap exception (quên truyền `cause`).

Kiến thức: exception hierarchy, `try-with-resources`, custom exception

🟪 PHẦN 3 — Java hiện đại (T4 28/10)

Bài 14 + 24 gộp — Inner class, Anonymous class, Lambda ⚡ ⭐⭐ 📅 T4 28/10

Interface `Validator<T>`: viết validate email bằng inner class, validate số điện thoại bằng anonymous class, validate tên bằng lambda. Sau đó dùng thử các functional interface có sẵn: `Predicate`, `Function`, `Consumer`, `Supplier`, và method reference `::`.

Lambda Java tương đương arrow function JS, chỉ khác là phải gắn với một functional interface.

Kiến thức: functional interface, lambda, method reference

Bài 15 — Record ⚡ ⭐⭐ 📅 T4 28/10

Tạo `record Point(double x, double y)` và `record Person(String name, int age, String email)`; dùng compact constructor để validate.

Kiến thức: `record`, immutability (sẽ dùng làm DTO trong Spring Boot)

Bài 16 — Sealed + Pattern matching 🖐 ⭐⭐⭐ 📅 T4 28/10

Mô hình hóa kết quả gọi API (dùng Java 21):

```
sealed interface ApiResult<T> permits Success, Failure, Loading {}
record Success<T>(T data) implements ApiResult<T> {}
record Failure<T>(String error, int code) implements ApiResult<T> {}
record Loading<T>() implements ApiResult<T> {}
```

Viết `render(ApiResult<User> r)` dùng `switch` pattern matching với record pattern, **không có nhánh `default`**. Thử thêm một subtype mới để thấy compiler báo lỗi.

Liên hệ: giống discriminated union trong TypeScript.

Kiến thức: `sealed`, `permits`, record pattern, exhaustive switch

🟨 PHẦN 4 — Collections & Generics (T5 29/10)

Bài 17 + 22 gộp — List & Set ⚡ ⭐⭐ 📅 T5 29/10

Thao tác `ArrayList` (thêm, xóa, tìm, sắp xếp). Lọc trùng bằng `LinkedHashSet`. So sánh `HashSet`, `LinkedHashSet` và `TreeSet` về thứ tự phần tử. Tạo list chỉ đọc bằng `List.of()` và thử sửa nó.

Câu cần tự trả lời được: *`ArrayList` khác `LinkedList` thế nào, khi nào dùng cái nào?*

Bài 18 — HashMap: Đếm tần suất từ 🖐 ⭐⭐ 📅 T5 29/10

Nhập một đoạn văn, đếm số lần xuất hiện của mỗi từ. In top 5 từ phổ biến nhất và các từ chỉ xuất hiện 1 lần. Dùng `merge()` hoặc `getOrDefault()`.

Câu cần tự trả lời được: *HashMap hoạt động bên trong thế nào (bucket, hash collision)? Vì sao key phải có `hashCode` ổn định?*

> **Bẫy hay gặp:** tách từ bằng `split(" ")` thay vì `split("\\s+")`; quên chuyển chữ thường và loại bỏ dấu câu.

Bài 20 — PriorityQueue: Hàng đợi bệnh viện 🔍 ⭐⭐⭐ 📅 T5 29/10

Bệnh nhân có tên và mức ưu tiên (EMERGENCY, HIGH, MEDIUM, LOW). Dùng `PriorityQueue` để luôn xử lý bệnh nhân ưu tiên cao nhất trước. **Thêm yêu cầu:** hai bệnh nhân cùng mức ưu tiên thì ai đến trước được xử lý trước.

Khi review code AI viết, kiểm tra xem nó có giữ đúng thứ tự đến khi cùng mức ưu tiên không. `PriorityQueue` không đảm bảo điều này nếu không có tiêu chí phụ.

Bài 23 — Deque: Lịch sử trình duyệt 🔍 ⭐⭐⭐ 📅 T5 29/10

Dùng `ArrayDeque` cài đặt `visit(url)`, `back()`, `forward()`, `printHistory()`. Khi visit trang mới thì xóa lịch sử forward.

Viết test cho các edge case: `back()` khi đang ở trang đầu tiên, visit sau khi đã back nhiều lần.

> Bài 19 (Generic Stack) và bài 21 (Map lồng nhau): **bỏ**. Generics đã được luyện ở bài 32; Map lồng nhau đã có trong bài 26–27.

🟧 PHẦN 5 — Stream API (30/10 – 02/11)

Bài 25 — Stream với số ⚡ ⭐⭐ 📅 T6 30/10

Với `List<Integer>` từ 1 đến 50: lọc số nguyên tố, tính tổng bình phương các số lẻ, lấy top 5 sau khi nhân 3, thử `anyMatch`, `allMatch`, `reduce`.

*Liên hệ: giống `filter` / `map` / `reduce` của mảng JS, nhưng **lazy** (chưa chạy cho đến khi gọi thao tác kết thúc) và **chỉ dùng được một lần**.*

Bài 26 — Stream với Object 🖐 ⭐⭐⭐ 📅 T6 30/10

Class `Employee` (name, department, salary, yearsOfExperience), tạo danh sách từ 20 nhân viên trở lên. Dùng Stream để:

1. Lọc nhân viên lương trên 15 triệu và kinh nghiệm từ 3 năm
2. Tính lương trung bình theo phòng ban
3. Lấy top 3 nhân viên lương cao nhất
4. Nhóm nhân viên theo phòng ban (`groupingBy`)
5. Đếm số nhân viên mỗi phòng ban

Mẹo: nhờ AI sinh dữ liệu mẫu 20 nhân viên. Phần đó không đáng để tự gõ.

Bài 27 — Collectors nâng cao 🖐 ⭐⭐⭐ 📅 T6 30/10

Tiếp tục bài 26: `partitioningBy` (lương trên/dưới 20 triệu), `joining` (ghép tên theo phòng ban), `toMap` có xử lý key trùng, `summarizingDouble` (thống kê lương).

> **Bẫy hay gặp:** dùng `toMap` khi có key trùng sẽ ném `IllegalStateException`; dùng `double` cho tiền (trong thực tế dùng `BigDecimal`).

Bài 28 — Optional 🔍 ⭐⭐⭐ 📅 T2 02/11

`UserRepository.findByEmail()` trả về `Optional<User>`. Dùng `orElse`, `orElseGet`, `orElseThrow`, `map`, `flatMap`, `filter`.

Khi review, tìm các cách dùng Optional sai: gọi `.get()` mà không kiểm tra, dùng Optional làm field hoặc tham số, dùng `orElse(taoObjectTonKem())` thay vì `orElseGet`.

Bài 29 — Stream đọc file ⚡ ⭐⭐ 📅 T2 02/11

Đọc file text bằng `Files.lines()` trong `try-with-resources`: đếm dòng, từ, ký tự; tìm 10 từ dài nhất; lọc các dòng chứa từ khóa. Phần đếm tần suất giống bài 18, chỉ cần viết lại bằng Stream.

Bài 30 — Phân tích dữ liệu bán hàng từ CSV 🖐 ⭐⭐⭐⭐ 📅 T2 02/11

File CSV gồm: ngày, sản phẩm, số lượng, giá. Parse file rồi dùng Stream để tính:

1. Tổng doanh thu theo từng tháng (`YearMonth`)
2. Sản phẩm bán chạy nhất
3. Ngày có doanh thu cao nhất
4. Trung bình số đơn hàng mỗi ngày
5. In báo cáo dạng bảng ra console

Dùng `record SaleRow(...)` cho mỗi dòng dữ liệu. Dòng lỗi định dạng thì bỏ qua và ghi log, không để chương trình dừng.

> **Bẫy hay gặp:** tính tiền bằng `double` ; không xử lý dòng trống hoặc header; sai format khi parse `LocalDate` .

🟥 PHẦN 6 — Concurrency & JVM (MỚI) (03/11 – 04/11)

Bộ cũ thiếu phần này, trong khi đây là chủ đề phỏng vấn backend rất hay hỏi và cũng là nơi code AI hay sai nhất.

Bài 34 — Race condition & Deadlock 🖐 ⭐⭐⭐ 📅 T3 03/11

1. Tạo `Counter` có method `increment()` . Chạy 1000 thread, mỗi thread tăng 1000 lần. Quan sát kết quả bị sai (khác 1.000.000).
2. Sửa theo 3 cách: `synchronized` , `AtomicInteger` , `ReentrantLock` . Đo thời gian chạy của từng cách.
3. Lấy lại `BankAccount` từ bài 8, viết `transfer(from, to, amount)` . Cho 2 thread chuyển tiền chéo nhau (A→B và B→A) để tạo ra **deadlock**, sau đó sửa bằng cách luôn khóa theo thứ tự id tài khoản.

Kiến thức: `Thread` , `synchronized` , `AtomicInteger` , `ReentrantLock` , deadlock

Bài 35 — ExecutorService 🖐 ⭐⭐⭐ 📅 T3 03/11

1. Giả lập tải 20 file, mỗi file `Thread.sleep()` ngẫu nhiên 200–800ms. So sánh thời gian chạy tuần tự với chạy bằng `Executors.newFixedThreadPool(4)` .
2. Dùng `Future` , `invokeAll` , và shutdown đúng cách (`shutdown` + `awaitTermination`).
3. Viết lại bài 18 theo kiểu đa luồng: chia văn bản thành nhiều phần, đếm song song, gộp kết quả bằng `ConcurrentHashMap.merge()` .
4. Thử thêm **virtual thread** (Java 21): `Executors.newVirtualThreadPerTaskExecutor()` .

Kiến thức: thread pool, `Future` , `ConcurrentHashMap` , virtual thread

Bài 36 — CompletableFuture (≈ Promise của JS) 🔍 ⭐⭐⭐ 📅 T4 04/11

Giả lập 3 API: `getUser()` , `getOrders()` , `getRecommendations()` , mỗi cái sleep khoảng 500ms.

1. Gọi cả 3 song song, gộp kết quả thành `Dashboard` bằng `thenCombine` hoặc `allOf` (giống `Promise.all`).
2. Nếu `getRecommendations()` lỗi thì trả về danh sách rỗng (`exceptionally`); nếu chậm quá 1 giây thì dùng `orTimeout` hoặc `completeOnTimeout` .
3. Khi review code AI viết, kiểm tra: có chỗ nào gọi `.join()` hoặc `.get()` quá sớm làm mất tác dụng chạy song song không? Lỗi có bị nuốt mất không?

Kiến thức: `CompletableFuture` , `thenApply` / `thenCompose` / `thenCombine` , xử lý lỗi bất đồng bộ

Bài 37 — JVM và những cái bẫy ⚡ ⭐⭐ 📅 T3 03/11

Viết từng đoạn code nhỏ, **đoán kết quả trước** rồi mới chạy:

1. `Integer a = 127, b = 127; a == b` so với `Integer a = 128, b = 128; a == b` (Integer cache)
2. `"abc" == "abc"` so với `new String("abc") == "abc"` (String pool)
3. Gọi đệ quy không có điểm dừng → `StackOverflowError`; chạy với `-Xmx16m` rồi liên tục thêm phần tử vào list → `OutOfMemoryError`
4. Nối String bằng `+` trong vòng lặp 100.000 lần so với dùng `StringBuilder`, đo thời gian
5. Truyền object vào method rồi gán lại tham số: vì sao object bên ngoài không đổi? (Java luôn truyền theo giá trị)

Kiến thức: heap và stack, String pool, GC ở mức cơ bản, pass-by-value

🎯 PHẦN 7 — Tổng hợp (04/11 – 06/11)

Bài 32 — LRU Cache 🖐 ⭐⭐⭐⭐ 📅 T4 04/11

Cài đặt `LRUCache<K, V>` với capacity cố định bằng `LinkedHashMap` (`accessOrder = true` + override `removeEldestEntry`).

Mở rộng (có thể để sau nếu không kịp): viết thêm một bản tự cài bằng `HashMap` + danh sách liên kết đôi (câu phỏng vấn kinh điển). Sau đó làm cache thread-safe và giải thích cách bạn chọn.

> **Bẫy hay gặp:** quên `accessOrder = true`, khiến cache hoạt động như FIFO chứ không phải LRU.

Bài 33 — Hệ thống bán hàng (dự án cuối) 🖐 + AI ⭐⭐⭐⭐ 📅 T5 05/11 – T6 06/11

Gồm `Product`, `Customer`, `Order`, `Store`: CRUD sản phẩm, đặt hàng (trừ tồn kho), hủy đơn (hoàn tồn kho), thống kê doanh thu, top sản phẩm bán chạy. Dùng lại `OrderStatus` từ bài 12.

Làm theo quy trình như đi làm thật với AI:

1. **[05/11] Tự viết spec** (khoảng nửa trang): các entity, nghiệp vụ, quy tắc (ví dụ: không hủy được đơn đã giao; tồn kho không được âm).
2. **[05/11] Tự code** phần lõi nghiệp vụ (đặt hàng, hủy đơn).
3. **[05/11]** Để AI viết phần phụ (dữ liệu mẫu, in báo cáo) và **review từng thay đổi** trước khi nhận.
4. **[06/11]** Viết 8–10 test JUnit 5. Tự nghĩ test case, có thể để AI viết code test, nhưng các case sau phải có: hủy đơn 2 lần, đặt hàng vượt tồn kho, 2 thread cùng đặt món hàng cuối cùng.
5. **[06/11]** Tạo project bằng Maven, đẩy lên GitHub, README có phần *"Những quyết định thiết kế và lý do"*.

Kiến thức: tổng hợp toàn bộ Java Core, Maven, JUnit, quy trình làm việc cùng AI

📅 Lịch 10 ngày (26/10 – 06/11/2026)

Ngày	Bài	Trọng tâm
T2 26/10	1–6 (gộp), 7	Cú pháp, ghi chú Java ≠ JS, `equals` / `hashCode`
T3 27/10	8–10, 11, 13	Kế thừa, interface, Comparator, exception
T4 28/10	12, 14+24, 15, 16	Enum, state machine, lambda, record, sealed + pattern matching

T5 29/10	17+22, 18, 20, 23	Collections, HashMap
T6 30/10	25, 26, 27	Stream, Collectors
T7–CN 31/10 – 01/11	*Nghỉ*	*Không bắt buộc: làm lại 1 bài 🖐 bất kỳ mà không xem code cũ*
T2 02/11	28, 29, 30	Optional, đọc file, phân tích CSV
T3 03/11	34, 35, 37	Race condition, deadlock, thread pool, JVM
T4 04/11	36, 32	CompletableFuture, LRU Cache
T5 05/11	33 (phần 1)	Dự án cuối: spec, code lõi nghiệp vụ, Maven
T6 06/11	33 (phần 2) + tự kiểm tra	JUnit, GitHub, README; checklist bên dưới, chuẩn bị sang Spring Boot
T7–CN 07/11 – 08/11	*Nghỉ*	*Dự phòng nếu bị chậm*

MẸO

Bài không được bỏ: 7, 11, 13, 18, 26, 27, 30, 32, 33, 34. Nếu bị chậm tiến độ, giữ các bài này và chuyển các bài ⚡ thành chỉ đọc lời giải. Ngày nặng nhất là 02/11 (bài 30) và 04/11 (bài 32): nếu đuối, chuyển bài 29 thành chỉ đọc lời giải và để phần mở rộng của bài 32 sang sau.

✅ Checklist tự kiểm tra (T6 06/11)

Giải thích được bằng lời, không xem tài liệu. Có thể nhờ AI đóng vai người phỏng vấn hỏi xoáy từng câu.

☐ Hợp đồng `equals` / `hashCode` là gì; điều gì xảy ra khi vi phạm
☐ `HashMap` hoạt động bên trong thế nào; vì sao key nên là immutable
☐ `ArrayList` khác `LinkedList`; `HashSet` khác `TreeSet` khác `LinkedHashSet`
☐ Checked khác unchecked exception; khi nào tạo custom exception
☐ Abstract class khác interface; `record` dùng để làm gì
☐ Stream lazy nghĩa là gì; `map` khác `flatMap`
☐ Những cách dùng `Optional` sai
☐ Race condition là gì; `synchronized` khác `AtomicInteger`; cách tránh deadlock
☐ Vì sao dùng thread pool; virtual thread giải quyết vấn đề gì
☐ Heap khác stack; String pool; Java truyền tham số theo giá trị nghĩa là gì

➡️ Sau 2 tuần (từ T2 09/11)

Chuyển sang Spring Boot + SQL. Khi đó, những thứ bạn vừa học sẽ xuất hiện trở lại:

Java Core	Gặp lại trong Spring Boot
Interface, polymorphism	Dependency injection, tầng Service/Repository
Record	DTO cho request/response
Exception	`@ControllerAdvice`, xử lý lỗi API

Enum làm state machine (bài 12)	Trạng thái của entity
Optional (bài 28)	`findById()` của Spring Data JPA
Concurrency (bài 34–36)	Transaction, `@Async`, xử lý request đồng thời
Dự án bài 33	Chuyển thành REST API + PostgreSQL, rồi nối với frontend của bạn